Smooth and Nonsmooth Markowitz Portfolio Optimization

Lucas Ahou

Guerand Dewell

I. Introduction

Portfolio optimization is the core of modern quantitative finance. It helps investor solve the trade-off problem between maximizing expected returns from investments while managing risks. Originally, this problem was described as a smooth mean-variance problem introduced by Markowitz [1]. This problem minimizes the portfolio variance while targeting a certain expected return. This first problem, however, does not take *transaction cost* into account, therefore leading to a second version of this problem integrating this in the objective. This new problem is more realistic, though it is now non-smooth. Because of this key difference between the two models, the suitable optimization methods will differ from one another.
The aim of this report is thus to compare different methods for both models by analyzing the computational cost of the methods and their convergence, both empirically and using theoretical results.

II. Data

For both models, we will build them based on historical data sampled from the S&P500 index. The dataset is described in a .csv file where we have access to the name of the different stocks, the date and the open, close and low/high prices at that date[1]. We are actually only interested in the date, name and close prices in this dataset. In fact we are actually interested in finding the average return vector μ and the corresponding covariance matrix of returns Σ. The first step was to extract the close prices as a matrix where each row corresponds to a different date and each column corresponds to a different stock's name. We then had to compute the returns at each date. At a time t, the return r_t is given by:

$$r_t = (p_t - p_{t-1})/p_{t-1}$$

The return μ is then simply given by the average of the returns with respect to time, for each assets. The covariance matrix Σ is also simply given by the covariance matrix of the return matrix. A strong property of (sample) covariance matrices is that they are always square, symmetric and positive semi-definite. This property is mandatory to be able to use strong theoretical results for convergence because it makes our objective function convex. In our implementation, we also included a feature to select only n stocks within all the available stocks. This will allow us to compare the computational costs of the diverse methods as the dimension of the variable increases.

III. Smooth Model

The first model we will study is the *Smooth Markowitz model*. In this section, we will first present the formulation of the model and the meaning of its constituting parts. We will then present the three methods we will implement to solve this problem and that we will analyze later.

A. Model

The *Smooth Markowitz model* is a mean-variance problem that was introduced by Markowitz in 1952[1]. It is defined as follows:

$$\min_{w \in \Delta} f(w) = \frac{1}{2} w^\top \Sigma w - \lambda w^\top \mu \tag{1}$$

where $w \in \mathbb{R}^n$, $\Sigma \in \mathbb{R}^{n \times n}$ and $\mu \in \mathbb{R}^n$.
The vector of variables w represents the *weights* of our portfolio, i.e. the proportion of each assets that constitutes it. Because each component w_i of w represents a percentage of our portfolio, they must sum up to one in a realistic scenario. This is why we define the feasible set as the simplex:

$$\Delta = \left\{ w \in \mathbb{R}^n : w_i \geq 0, \mathbb{1}^\top w = 1 \right\}$$

The matrix Σ is the covariance matrix and μ is the average vector of the returns available in the available dataset. Therefore, the model aims to do the following:

1. The first term $\frac{1}{2} w^\top \Sigma w$ represents the variance of the portfolio's return. Intuitively, we want to minimize it as this is a measure of risk. In fact, a highly variating portfolio return we will often encounter very high returns, as well as very low ones which indicates an unstable portfolio. Thus minimizing this term allows to have more consistent returns.
2. The second term $-\lambda w^\top \mu$ is there to target a maximum average return while still minimizing the variance with the presence of the first term. Without this, the solution for this problem will only try to aim for consistent returns and will thus not make a lot of profit. The $\lambda > 0$ constant is a hyperparameter which controls the risk. The bigger it is, the more the model will try to find a portfolio selection that maximizes the average return compared to minimizing the variance. Therefore, this parameter represents the risk we are willing to take before solving this problem. The greater it is, the higher the risk is.

Now that we have seen an overview of this model and explained its meaning, we will discuss some properties of this model.
First of all, the most important aspect is that the model is convex. In fact, the objective function is a sum between a positive semi-definite quadratic form (first term) and a linear function in the variables (second term). A sum of two convex functions is convex, so the objective is convex. The feasible set is also convex. This can be easily checked with the following computations:

Let $w, v \in \Delta = \left\{ w \in \mathbb{R}^n : w_i \geq 0, \sum_{i=1}^n w_i = 1 \right\}$, $\gamma \in [0,1]$ and $u := \gamma w + (1-\gamma) v$:

1. $u_i = \gamma w_i + (1-\gamma) v_i \geq 0$ because $w_i, v_i, \gamma, (1-\gamma) \geq 0$ ✓
2. $\sum_i u_i = \sum_i \gamma w_i + (1-\gamma) v_i = \gamma \sum_i w_i + (1-\gamma) \sum_i v_i = \gamma + (1-\gamma) = 1$ ✓

Thus $u \in \Delta$ and Δ is convex ■

Proof 1. Δ is convex

[1]Note that the data only contains business days so two consecutive dates are not necessarily consecutive days.

The reason why it is important is that the local minima of a convex problem is also **global**. This will ensure that every method that we implement will converge towards a minimum with the same objective value. It will thus not be necessary to compare the performance of the portfolio in terms of its returns. Another important aspect of convex problems is that it allows us to use strong properties of convergence for the different methods.

Regarding those results and the implementation of the methods, we need to discuss some key properties of this model. More precisely, we need to derive the gradient and the hessian of the objective function, its smoothness constant as well as the projection operator on the simplex

a) *Gradient and hessian of f:* We will simply differentiate the objective function f with respect to w to obtain the gradient:

$$\nabla f(w) = \Sigma w - \lambda \mu$$

and a second time to get the hessian:

$$\nabla^2 f(w) = \Sigma$$

Something to notice is that the computational complexity of a gradient evaluation is $\mathcal{O}(n^2)$ because it is a matrix-vector multiplication. However, in the context of the *coordinate descent* method, it is not needed to compute the whole gradient as we will see later.

b) *Smoothness constant:* The smoothness constant of f is needed for the *Projected Gradient Descent* for example to use a step size that gives us better guarantees of convergence. Here is the derivation of this constant:

$\forall w, v \in \mathbb{R}^n$, we have:

$$\begin{aligned} \|\nabla f(w) - \nabla f(v)\|_2 &= \|\Sigma w - \Sigma v\|_2 \leq \|\Sigma\| \cdot \|w - v\|_2 \\ &= |\lambda_{\max}(\Sigma)| \cdot \|w - v\|_2 \\ \implies L &= |\lambda_{\max}(\Sigma)| = \lambda_{\max}(\Sigma) \end{aligned}$$

where we used the fact that Σ is PSD to simplify the last expression.

Derivation 1. Smoothness constant

c) *Projection on the simplex:* The projection on the simplex is used by every method presented in this report, except for the interior point method. Below, we derive the calculation of the projection on this set:

We want to solve:

$$P_\Delta(v) = \arg\min_{w \in \Delta} \|w - v\|_2^2$$

which can be formulated as a constrained quadratic minimization problem:

$$\min_{w \in \mathbb{R}^n} \frac{1}{2} \|w - v\|_2^2 \quad \text{s.t.} \quad \sum_{i=1}^{n} w_i = 1, w_i \geq 0$$

We now define the Lagrangian of this problem as follows:

$$\mathcal{L}(w, \theta, \alpha) = \frac{1}{2} \sum_i (w_i - v_i)^2 - \theta \left(\sum_i w_i - 1 \right) - \sum_i \alpha_i w_i$$

where $\theta \in \mathbb{R}$ and $\alpha \in \mathbb{R}^n$ are the Lagrange multipliers associated with the equality and nonnegativity constraints respectively.

Derivation 2. Projection operator on the simplex

We now impose the KKT conditions [2]:

1. Stationarity:
$$\frac{\partial \mathcal{L}}{\partial w_i} = w_i - v_i - \theta - \alpha_i = 0 \Rightarrow w_i = v_i + \theta + \alpha_i$$
2. Primal feasibility:
$$w_i \geq 0, \quad \sum_{i=1}^{n} w_i = 1$$
3. Dual feasibility:
$$\alpha_i \geq 0$$
4. Complementary slackness:
$$w_i \alpha_i = 0$$

and we now have to solve them. First, we notice that from the complementary slackness:

$$\begin{aligned} &\text{If } w_i > 0, \text{then } \alpha_i = 0 \Rightarrow w_i = v_i + \theta \\ &\text{If } w_i = 0, \text{then } \alpha_i \geq 0 \Rightarrow v_i + \theta \leq 0 \end{aligned}$$

The second case simply shows that we do not violate any KKT conditions. We thus have:

$$w_i = \max(v_i + \theta, 0)$$

To determine θ, we use the equality constraint:

$$\sum_i \max(v_i + \theta, 0) = 1$$

This sum can also be expressed in the following manner:

$$\sum_{i: v_i + \theta > 0} v_i + \theta = 1$$

If we now define k as the number of indices that satisfy $v_i + \theta > 0$ and $v_{(i)}$ be the sorted components of v, we get:

$$\sum_{i: v_i + \theta > 0} v_i + \theta = \sum_{i=n-k+1}^{n} v_{(i)} + \theta = \sum_{i=n-k+1}^{n} v_{(i)} + k\theta$$

This gives us:

$$\theta = \frac{1 - \sum_{i=n-k+1}^{n} v_{(i)}}{k}$$

This k also needs to satisfy:

$$v_{(n-k)} + \theta \leq 0 \quad \text{and} \quad v_{(n-k+1)} + \theta > 0$$

There is a unique k that satisfies this relation which gives us the correct theta.

Derivation 2. Projection operator on the simplex (cont.)

The number k–and thus the projection–can be implemented more efficiently using this algorithmic approach:

Input: Vector v to project
Output: $P_\Delta(v)$
Time complexity: $\mathcal{O}(n \log n)$

1. Sort v in descending order $\Rightarrow u_1 \geq u_2 \geq \ldots \geq u_n$
2. $k \longleftarrow \max\left\{ j \in \{1, \ldots, n\} : u_j + \frac{1 - \sum_{i=1}^{j} u_j}{j} > 0 \right\}$
3. $\theta \longleftarrow \frac{1}{k}\left(1 - \sum_{i=1}^{k} u_j\right)$
4. **return** w with $w_i = \max(v_i - \theta, 0)$

Algorithm 1. Projection on the simplex

B. Projected Gradient Descent

We will now describe the first optimization method we will implemented for this model: the Projected Gradient Descent. Given a point $w \in \Delta$, the idea is simply to perform a gradient descent step:

$$z = w - \alpha \nabla f(w)$$

where $\alpha > 0$ is our step size. However, it is clear that there is no guarantee that this new point z stays in the simplex after the step. We therefore will simply project this point on the simplex using Algorithm 1:

$$w^{+} = P_{\Delta}(z) = P_{\Delta}(w - \alpha \nabla f(w))$$

The complete algorithm is described as follows:

Input: $w_0, \alpha > 0$
Output: approximate solution w_N

for $k = 0, 1, ..., N-1$ **do**
  $g_k \longleftarrow \nabla f(w_k)$
  $w_{k+1} \longleftarrow P_{\Delta}(w_k - \alpha g_k)$
end for

Algorithm 2. Projected Gradient Descent

Here (as well as for the other methods) multiple stopping criterions can be used. We can either, as described above, stop after a certain number of iterations, or we could stop after reaching a certain precision (either on the objective value as well as the iterate values). Altough we do not have theoretical results that can improve the convergence of the method, we have actually a lower-bound on the number of iterations to reach a certain precision if we take the right step size. In fact, here we have a smooth objective function f with a smoothness constant $L = \lambda_{\max}(\Sigma)$. In this case, we can take the step size to be:

$$\alpha = \frac{1}{L}$$

Knowing that our function is also convex, we have the following convergence result:

$$f(w_k) - f^\star \leq \frac{L\,\|w_0 - w^\star\|^2}{2k} \leq \frac{L}{k}, \quad \forall k \geq 1$$

where we used the fact that $\|w_0 - w^\star\|^2 \leq D^2 = 2$ where D is the diameter of the simplex.

Hence, our method has a rate of $\mathcal{O}(1/k)$, which also means that we need $\mathcal{O}(1/\varepsilon)$ to reach an ε-accuracy for the objective value. We will later confirm this rate numerically.

C. Adaptive step for Projected Gradient Descent

The projected gradient method we just presented used a fixed step size. We can however improve the convergence of this method by using *adaptive steps*. In this section, we will present three adaptive step sizes that we will implement to, hopefully, obtain better performances. Note, however, that those step size were originally designed for unconstrained problem. In practice, they also work quite well in the projected case as we will see later with numerical results.

a) *Armijo backtracking line search:*

The first adaptive step size method we will implement is the *Armijo Line Search*. This method starts with a candidate new iterate and decreases the step size until a condition is satisfied. The algorithm is described as follows:

Input: $w_k, c \in (0,1), \rho \in (0,1), \alpha_0 > 0$
Init: $k := 0$
Output: Step size α_k for PGD

Step 1: $w_k^{+} := P_{\Delta}(w_k - \alpha_k \nabla f(w_k))$
Step 2:
- If $f(w_k^{+}) \leq f(w_k) - c\alpha_k\,\|\nabla f(w_k)\|^2$: **return** α_k
- Else: Set $\alpha_{k+1} := \rho\alpha_k, k := k+1$, Go to **Step 1**

Algorithm 3. Armijo Line Search for PGD

This method has several advantages. First of all, it does not require the Lipschiz constant L of the gradient. Also, it offers the same computational complexity as the original projected gradient descent. Most importantly, it often considerably increases the convergence of this method.

b) *Barzilai-Borwein step size:*

Until now, we never used second order information in the model. We may be tempted to implement the Newton's method instead of a Gradient Descent but it requires computing the inverse of the hessian Σ. In practice, it is expensive and sometimes (likely to be the case here) the hessian is singular. The Barzilai-Borwein method, applied to the Gradient Descent, aims to find a step size α_k that approximates the Newton step. Here is the description of this method:

Input: w_k, w_{k-1}
Output: Step size $\alpha_k := \frac{s^{(k-1)^\top} s^{k-1}}{s^{(k-1)^\top} y^{k-1}}$
where $s^{k-1} := w^k - w^{k-1}, y^{k-1} := \nabla f(w_k) - \nabla f(w_{k-1})$

Algorithm 4. Barzilai-Borwein Method (Least-Square version)

Notice that this step size is not defined for the first iteration when $k = 0$. In this case, we just choose $\alpha_0 = \frac{1}{L}$ to perform a regular gradient descent before using the Barzilai-Borwein step for the following iterations.

c) *Exact line search:*

Both adaptive step method we showed previously were *inexact* step size. Here we will take a look at the *exact* step size. Here is how it is derived:

Consider the objective value at the new iterate:

$$f(x_{k+1}) = f(x_k - \alpha_k \nabla f(x_k)) := J(\alpha_k)$$

This is a function of the step size and can thus be minimized accordingly. After minimizing, we obtain the following expression:

$$\alpha_k = \frac{\nabla f(x_k)^\top \nabla f(x_k)}{\nabla f(x_k)^\top \Sigma \nabla f(x_k)}$$

As said previously, the covariance matrix Σ may contain zero (or close to zero) eigenvalues. The quadratic form at the denominator can therefore be null in some cases. To avoid that in practice, we will precompute the denominator and we use $\alpha_k = \frac{1}{L}$ if it is smaller than a certain tolerance.

D. Projected Gradient Descent with Momentum

Here we are going to slightly modify the previous algorithm by introducing the notion of momentum. In our previous algorithm, we were not taking advantage of the gradient of the previous iterates, i.e. $\nabla f(w_{k-1}), \nabla f(w_{k-2}), ..., \nabla f(w_0)$. We could then introduce a momentum variable

$$m_{k+1} = \beta m_k + (1-\beta)\nabla f(w_k),$$

with $\beta \in [0,1]$ and $m_0 = 0$.

The projected momentum iterates becomes for an $w_k \in \Delta$

$$w_{k+1} = P_\Delta(w_k - \gamma m_{k+1})$$

Momentum increases the influence of recent gradients while gradually vanishing older ones, which often accelerates convergence. The complete algorithm is described as follows :

Input: w_0, γ, β
Output: approximate solution w_N

$m_0 = 0$
for $k = 0, 1, ..., N-1$ **do**
  $g_k \longleftarrow \nabla f(w_k)$
  $m_{k+1} = \beta m_k + (1-\beta) g_k$
  $w_{k+1} = P_\Delta(w_k - \gamma m_{k+1})$
end for

Algorithm 5. Projected Gradient Descent with Momentum

However, with this algorithm (especially with a fixed β), the rate of convergence has the same order $\mathcal{O}(1/k)$ as the projected gradient descent in the worst-case scenario.
A better version of this algorithm in the context of convex and smooth functions is the *Nesterov's Accelerated Gradient Method*, whose algorithm is described below:

Input: w_0, L
Init: $w_{-1} = w_0, \alpha = \frac{1}{L}, \lambda_0 = \beta_0 = 0$
Output: approximate solution w_N
for $k = 0, 1, ..., N-1$ **do**
  $y_k \longleftarrow w_k + \beta_k(w_k - w_{k-1})$
  $w_{k+1} \longleftarrow y_k - \alpha \nabla f(y_k)$
  $\lambda_{k+1} \longleftarrow \frac{1+\sqrt{1+4\lambda_k^2}}{2}, \quad \beta_{k+1} \longleftarrow \frac{\lambda_k - 1}{\lambda_{k+1}}$
end for

Algorithm 6. Nesterov's accelerated Gradient Method

This gives us this theoretical bound:

$$f(y_k) - f^* \leq \frac{2L \, \|w_0 - w^\star\|^2}{(k+1)^2}, \quad \forall k \geq 1$$

So our convergence rate is $\mathcal{O}(1/k^2)$, which is faster than gradient descent's one. This also implies that the number of iterations is $\mathcal{O}(1/\sqrt{\varepsilon})$ to reach a precision of ε on the objective value.

E. Projected Randomized Coordinate Descent

The last optimization method that we will implement for this model is the Projected Randomized Coordinate Descent. In this algorithm, rather than computing the full gradient, we are going to compute one component of the gradient. We are then going to perform one gradient descent in this direction. A description of the method can be written as follows:
Considering $w_k \in \Delta$, the step is defined by :

$$w_{k+1} = P_\Delta\Big(w_k - \alpha[\nabla f(w_k)]_{i_k}\Big)e_{i_k}$$

with $i_k \sim \mathcal{U}\{1, ..., n\}$
We directly identify several major issues. The first one occurs when we update our weight w_{k+1}. Indeed by proceeding in such way, we only update one component of the vector corresponding to the index i_k but projecting back onto the simplex induce a modification of the entire vector. To fix it we suggest two variants. The first one is the naive one as we are only going to rebalance our weight by modifying an other random weight such that $\sum_i w_i = 1$. The second one is a bit more complex. In this version we are also going to take two random weight and we are going to compute their gradient to direct ourself in the direction inducing the biggest variation in term of our objectif function for these two coordinates. The update is performed in such a way that the sum of the two selected weights is preserved, ensuring that the iterate remains in the simplex. The second issue is related to the stopping criterion from our algorithm. From the beginning we only considered the stopping criterion $\|w_{k+1} - w_k\| < \varepsilon$. However in the randomized coordinate descent, we may fall on a coordinate that won't move that much after the step. This will cause the algorithm to stop, even if the other coordinates are not optimized yet. Thus, the only stop criterion we will use is by applying a limit on the number of iterations.

IV. Non-smooth Model

A. Model

The second model we will study is a variant of the first one. Its formulation is given by:

$$\min_{w \in \Delta} f(w) = \frac{1}{2} w^\top \Sigma w - \lambda w^\top \mu + c\|w - w_{\text{prev}}\|_1 \tag{2}$$

In (2), we observe that the objective takes the same form as in (1). However, there is an additional non-smooth term. This term extends the previous model by taking transaction costs into account. In realistic scenarios, buying or selling assets implies a cost which is proportional to the amount of traded assets. To represent this in the model, we add a term proportional to the ℓ_1-distance between the current portfolio w and a reference one w_{prev}:

$$c\|w - w_{\text{prev}}\|_1$$

This essentially means that we do not want to drastically change the current allocation of our portoflio. In fact, the ℓ_1-norm encourages *sparse rebalancing*. The parameter c controls the trading cost. A small c implies that the portfolio can change drastically compared to a reference one and vice-versa.
The main difference with the smooth model is that we can not compute the gradient explicitly. Instead, we will have to use methods that either use a subgradient, or other optimization techniques.
In the next sections, we will present the three methods we will implement on this model, that is: projected subgradient method, proximal gradient descent and interior point method. For those methods, we will decompose the previous model into a smooth part and a non-smooth part which we will denote by:

$$g(w) := \frac{1}{2} w^\top \Sigma w - \lambda w^\top \mu, \quad h(w) := c\|w - w_{\text{prev}}\|_1$$
$$\Longrightarrow f(w) = g(w) + h(w)$$

B. Projected Subgradient Method

The subgradient method is simply a generalization of the gradient method for non-differentiable function where we can still get access to a subgradient. Let us first derive an expression for the subgradient:
The only non-smooth part of the objective function is h, we will therefore find a subgradient for this part. This function is not differentiable at $w = w_{\text{prev}}$. The subgradient is thus described by:

$$\partial h(w) = \begin{cases} -c & \text{if } w < w_{\text{prev}} \\ 0 & \text{if } w = w_{\text{prev}} \\ c & \text{if } w > w_{\text{prev}} \end{cases}$$

and the subgradient of f is simply the sum of the gradient of g and the subgradient of h.
We can now describe the following algorithm for the Projected Subgradient Method:

Input: w_0, α_k
Output: approximate solution w_N
for $k = 0, 1, ..., N-1$ **do**
  $w_{k+1} \longleftarrow P_\Delta(w_k - \alpha_k \partial f(w_k))$
end for

Algorithm 7. Projected Subgradient Method

[3] We will now discuss the step size for this method. If we denote $D := \max_{w,v\in\Delta} \|w - v\|_2$ the diameter of the simplex and $M \geq \|\partial f(w_k)\|$ the bound on the subgradient, we obtain the following inequality:

$$\min_{k=0,...,T} f(x_i) - f^\star \leq \frac{D^2 + M^2 \sum_{k=1}^T \alpha_k^2}{2\sum_{k=1}^T \alpha_k} \tag{3}$$

We now want to know for what sequence of α_k the righthand side of (3) is minimized. This term is convex and symmetric in $\alpha_1, ..., \alpha_T$ so the optimum is reached when all the α_k are equal ($\alpha_k := \alpha$). The right term thus simplifies to:

$$\frac{D^2 + M^2 T \alpha^2}{2T\alpha}$$

which is minimized when $\alpha = \frac{D}{M\sqrt{T}}$. With this step size, we now want to determine the value $T(\varepsilon)$ to obtain a certain precision ε. We have:

$$\min_{k=0,...,T} f(x_i) - f^\star \leq \frac{MD}{\sqrt{T(\varepsilon)}} \leq \varepsilon$$

We therefore have that $T(\varepsilon) \geq \frac{M^2 D^2}{\varepsilon^2} \sim \mathcal{O}(1/\varepsilon^2)$. Plugging that in the step size α:

$$\alpha \leq \frac{\varepsilon}{M^2}$$

This step size guarantees us to converge in $\mathcal{O}(1/\varepsilon^2)$ iterations to obtain an ε-accuracy solution. However, this convergence rate is quite bad. Even though this is the best we can theoretically do, we may want to choose a diminishing step size to, hopefully, obtain better performances in practice. Additionally to the constant step size we just presented, we will therefore also implement the following diminishing step size:

$$\alpha_k \leq \frac{\varepsilon}{M^2\sqrt{k}}$$

C. *Proximal Gradient Descent*

Instead of having to rely on the subgradient of h in our algorithm, we can use the *proximal gradient method*. We define the proximal operator for h given a step size t by:

$$\operatorname{prox}_{t,h}(x) = \arg\min_{z\in\mathbb{R}^n} \frac{1}{2t} \|x - z\|_2^2 + h(z)$$

The proximal gradient method is then given by (given w_0):

$$w_k = \operatorname{prox}_{t_k,h}\left(w_{k-1} - t_k \nabla g\left(x^{k-1}\right)\right), \quad k = 1, 2, ...$$

This method intuitively means:

- First term in the prox operator: find a point z that stays close to the gradient update of g
- Second term in the prox operator: Also, try to make h small

Even though this looks like we just reformulated the optimization problem, in some special cases the proximal operator has a closed-form expression. In our case, we have to obtain an expression for the proximal operator of the ℓ_1-norm. It can be proven that, for a function $\varphi(x) = c\|x\|_1$, the proximal operator is given by:

$$\left[\operatorname{prox}_{t,\varphi}(x)\right]_i = \begin{cases} x_i - ct & \text{if } x_i > ct \\ 0 & \text{if } -ct \leq x_i \leq ct \\ x_i + ct & \text{if } x_i < -ct \end{cases}$$

In our case, we have $h(w) = \varphi(w - w_{\text{prev}})$. It is easy to adapt the previous formula to this case
In the unconstrained case, given a fixed step size $t \leq \frac{1}{L}$ (where L is the smoothness constant of g), we have the following convergence result:

$$f(w_k) - f^\star \leq \frac{\|w_0 - w^\star\|_2^2}{2kt}$$

Therefore, this method has a convergence rate of $\mathcal{O}(1/k)$, or $\mathcal{O}(1/\varepsilon)$. However, we have to consider the cost of computing the proximal in practice. Fortunately, we have a variant of this method based on the Nesterov's accelerated gradient method, the *Accelerated Proximal Gradient Method*. This method is described by:

Input: w_0, L
Init: $y_1 = w_0, t_1 = 1$
Output: approximate solution w_N
for $k = 0, 1, ..., N-1$ **do**
  $x_k \longleftarrow \operatorname{prox}_{\frac{1}{L},h}\left(y_k - \frac{1}{L}\nabla f(y_k)\right)$
  $t_{k+1} \longleftarrow \frac{1+\sqrt{1+4t_k^2}}{2}$
  $y_{k+1} \longleftarrow w_k + \left(\frac{t_k - 1}{t_{k+1}}\right)(w_k - w_{k-1})$
end for

Algorithm 8. Accelerated Proximal Gradient Method

This time, in the unconstrained case, we have the following convergence result:

$$f(w_k) - f^\star \leq \frac{2\|w_0 - w^\star\|_2^2}{t(k+1)^2}$$

which thus gives us a convergence rate of $\mathcal{O}(1/k^2)$ or $\mathcal{O}(1/\sqrt{\varepsilon})$.

D. *Long-Step Path-Following Interior-Point method*

The *Long-Step Path-Following Interior-Point method* is a second-order method for solving convex optimization problems. The idea is to include the inequality constraints in the objective with barrier functions (often logarithmic barriers) and also introduce a *barrier parameter* which we will denote by t that multiplies the original objective function. We then solve this problem to obtain a solution $x^\star(t)$ which depends on the barrier parameter. The path $\{x^\star(t) : t > 0\}$ is called the *central path* and as $t \to \infty$, we have that $x^\star(t) \to x^*$. Given a generic convex barrier problem, the long-step variant of IPM is described as follows:

Input: $x_0, t_0, \tau \in (0,1), \theta \in (0,1)$
Output: approximate solution of x^*
while $\nu / t_k > \varepsilon$ **do**
  $t_{k+1} \longleftarrow t_k / (1 - \theta)$
  do *Damped Newton-Steps* **while** $\delta_{t_{k+1}}(x_{k+1}) > \tau$
  $k \leftarrow k + 1$
end while

Algorithm 9. Long-Step Path-Following IPM

In Algorithm 9, τ represents the target accuracy within each iteration, θ represents the scaling factor of the barrier parameter and ν is the self-concordant parameter of the barrier function. In the case of logarithmic barriers, this parameter is equal to the number of inequality constraints m. The algorithm also uses the *local norm* $\delta_t(x)$ which is defined by:

$$\delta_t(x) = \left(\nabla f_t(x)^\top [\nabla^2 f_t(x)]^{-1} \nabla f_t(x)\right)^{1/2}$$

Let's now try to adapt this method in the case of the non-smooth problem (2):
The original problem is described as:

$$\begin{aligned} \min_{w \in \mathbb{R}^n} \; & f(w) = \frac{1}{2} w^\top \Sigma w - \lambda w^\top \mu + c \|w - w_{\text{prev}}\|_1 \\ \text{s.t.} \; & \mathbb{1}^\top w = 1 \\ & w_i \geq 0 \quad \forall i = 1, ..., n \end{aligned}$$

Before transforming this formulation into a barrier problem, we need to get rid of the absolute values in the objective. To do that, we will introduce slack variables.
Let $u_i, v_i \geq 0$ such that $|w_i - w_{\text{prev},i}| = u_i + v_i$ and $w_i - w_{\text{prev},i} = u_i - v_i$. The problem now becomes:

$$\begin{aligned} \min_{x = (w,u,v) \in \mathbb{R}^{3n}} \; & \tilde{f}(x) := \frac{1}{2} w^\top \Sigma w - \lambda w^\top \mu + c \cdot \mathbb{1}^\top (u + v) \\ \text{s.t.} \; & \mathbb{1}^\top w = 1 \\ & w_i - w_{\text{prev},i} = u_i - v_i \quad \forall i = 1, ..., n \\ & w_i, u_i, v_i \geq 0 \quad \forall i = 1, ..., n \end{aligned}$$

Which gives us a new variable x of dimension $3n$, $m = 3n$ inequalities and $n+1$ equalities. Now, we can formulate the barrier problem. In order to do that, we will use logarithmic barriers to introduce the $3n$ inequality constraints in the objective function. We thus obtain:

$$\begin{aligned} \min_{x \in \mathbb{R}^{3n}} \; & \psi_t(x) := t \tilde{f}(x) - \sum_{i=1}^{n} \log(w_i) + \log(u_i) + \log(u_i) \\ \text{s.t.} \; & \mathbb{1}^\top w = 1 \\ & w_i - w_{\text{prev},i} = u_i - v_i \quad \forall i = 1, ..., n \end{aligned} \tag{4}$$

Note that, because we had $m = 3n$ inequality constraints, the self-concordant parameter of the barrier is $\nu = 3n$.
[4] We now have to derive the damped Newton-Steps for this problem. At each step, we will have to solve the Newton-KKT system to obtain the direction in which we will perform the step. This system is given by:

$$\begin{bmatrix} \nabla^2 \psi_t(x) & A^\top \\ A & \mathbf{0} \end{bmatrix} \begin{bmatrix} d_x \\ \beta_{\text{kkt}} \end{bmatrix} = - \begin{bmatrix} \nabla \psi_t(x) \\ Ax - b \end{bmatrix}$$

where d_x is the Newton-Step's direction. Here are the expression of the different parts of this system:

$$A = \begin{bmatrix} \mathbb{I}_n & -\mathbb{I}_n & \mathbb{I}_n \\ \mathbb{1}^\top & \mathbf{0} & \mathbf{0} \end{bmatrix}$$

$$\begin{cases} \nabla_w \psi_t(x) = t(\Sigma w - \lambda \mu) - \left[\frac{1}{w_1} \cdots \frac{1}{w_n}\right]^\top \\ \nabla_u \psi_t(x) = tc\mathbb{1} - \left[\frac{1}{u_1} \cdots \frac{1}{u_n}\right]^\top \\ \nabla_v \psi_t(x) = tc\mathbb{1} - \left[\frac{1}{v_1} \cdots \frac{1}{v_n}\right]^\top \end{cases} \Rightarrow \nabla \psi_t(x) = \begin{bmatrix} \nabla_w \psi_t(x) \\ \nabla_u \psi_t(x) \\ \nabla_v \psi_t(x) \end{bmatrix}$$

$$\begin{cases} H_w = t\Sigma + \operatorname{diag}\left\{\frac{1}{w_1^2}, ..., \frac{1}{w_n^2}\right\} \\ H_u = \operatorname{diag}\left\{\frac{1}{u_1^2}, ..., \frac{1}{u_n^2}\right\} \\ H_v = \operatorname{diag}\left\{\frac{1}{v_1^2}, ..., \frac{1}{v_n^2}\right\} \end{cases} \Rightarrow \nabla^2 \psi_t(x) = \operatorname{diag}(H_w, H_u, H_v)$$

To ensure feasibility of the Newton-Step and a strictly decreasing objective value, we have to damp the Newton steps. Choosing a damping of $\frac{1}{1+\delta(x)}$ satisfies those two properties. The Newton step thus becomes:

$$x_{k+1} = x_k + \frac{1}{1 + \delta(x)} d_x$$

To compute the local norm $\delta(x)$, we do **not** have to inverse the hessian. In fact, because the Newton step direction d_x satisfies at x_k:

$$d_x = -[\nabla^2 \psi_t(x_k)]^{-1} \nabla \psi_t(x_k)$$

We can simply compute:

$$\delta_t(x_k) = \left(-\nabla \psi_t(x_k)^\top d_x\right)^{1/2}$$

V. Numerical results

A. Smooth Model

Before diving into our numerical analysis, note that the optimal value of our objective function $f^\star$ was precomputed using a solver from the library cvxpy both in the smooth case as in the non-smooth case.
a) *Projected Gradient descent:*
The first experiment we performed was to compute the empirical number of iterations to achieve an ε-precision solution for many value of ε.

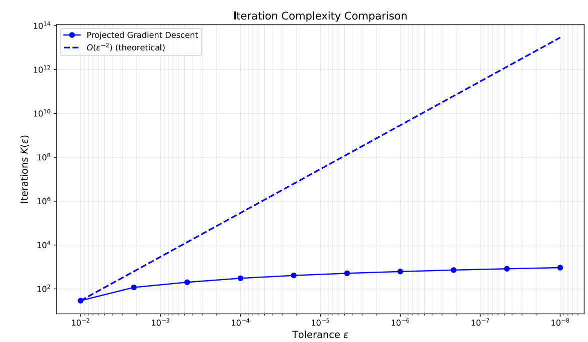


Fig. 1. Number of iterations vs. ε for PGD

As we can see on Fig. 1, the complexity of the PGD method is way below the theoretical one. In fact, it even looks like that the iteration complexity is of order $\mathcal{O}(1/\sqrt{\varepsilon})$, which shows that this method is performing way better than expected.
Now as we have mentionned previously, we have tried several adaptive step sizes. Here are the results we obtained :

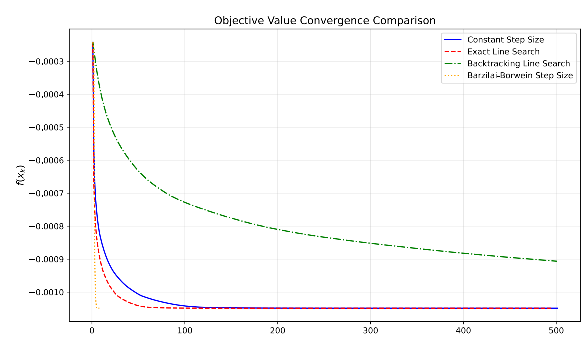
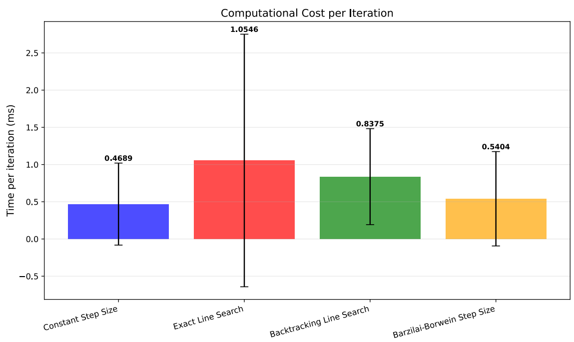


Fig. 2. $f(x_k)$ vs. k for the different adaptive step sizes ($\varepsilon = 10^{-8}$)

From Fig. 2, the adaptive step size method offering the best performance in terms of convergence is the *Barzilai-Borwein* adaptive step. Note, however, that it induces a bigger mean time per iteration than the fixed step size version. Despite the time per iteration being bigger, we only required 7 iterations to converge which mean that the total time is still way less for this adaptive step size than for the other. To illustrate this, we registered the time per iteration for each version to study the mean and the variance. Here are the results :



	Mean [ms]	Std [ms]	Iterations
Constant step size	0.2044	0.4038	500
Exact Line search	0.5080	1.6201	493
Backtracking Line Search	0.4163	0.4932	500
Barzilai-Borwein Step Size	0.4286	0.4950	7

Fig. 3. Time per iteration statistics ($\varepsilon = 10^{-8}$, max_iter = 500)

On Fig. 3, we can further see how the *Barzilai-Borwein* step clearly improves the convergence of this method.

b) *Projected Gradient descent with momentum:*

For this method, we will compare both versions of the *PGD with Momentum* presented previously which are the classic one and the Nesterov Accelerate version.

We will first analyze, as before, the number of iterations to reach convergence for different ε and compare both methods.

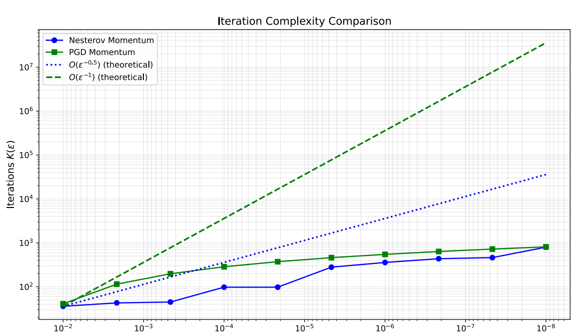


Fig. 4. Number of iterations vs. ε for PGD with Momentum

On Fig. 4, we see that both curves are strictly below their respective theoretical rate. We also notice that the Nesterov's method slightly better than the classic momentum no matter the value of ε. To further compare those methods, we will also look at the value of the objective function as a function of the iterations.

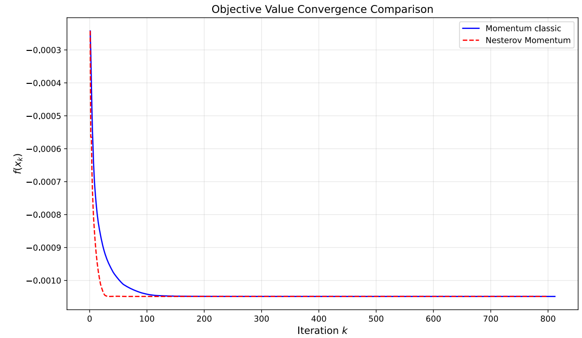


Fig. 5. $f(x_k)$ vs. k for Momentum and Nesterov's Momentum ($\varepsilon = 10^{-8}$)

Again, on Fig. 5 we clearly see that the Nesterov's Momentum converges faster than the classic one.

c) *Projected Randomized Coordinate Descent:*

For this method, we will take a look at the difference between the objective value at step k and the optimal value $f^\star$

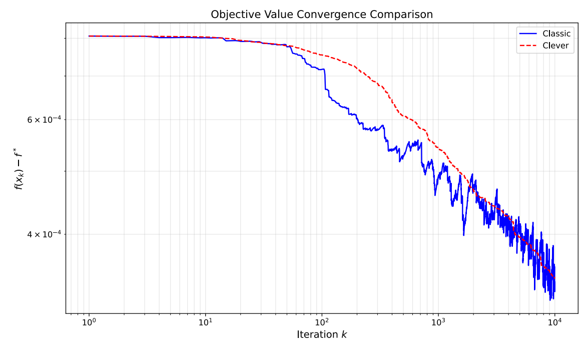


Fig. 6. $f(x_k) - f^\star$ vs. k (max_iter $= 10^4$)

On Fig. 6, we compared both implementations desribed previously. We can observe that, even after 10^4 iterations, none of the two methods reaches a good approximation for the optimal value. This is likely due to the projection step which causes a slow convergence. From this graph, we can conclude that this method is not suited to this problem.

B. *Comparison of the model :*

We will now compare all the methods shown above by plotting the objective value to observe the convergence.

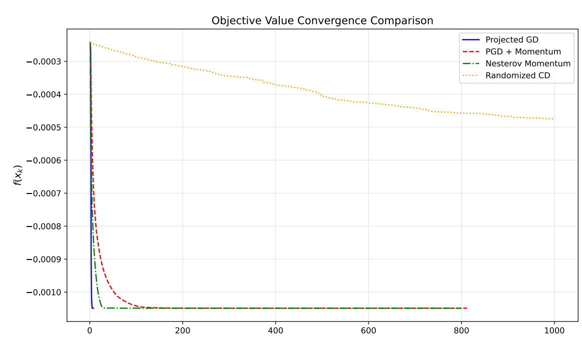
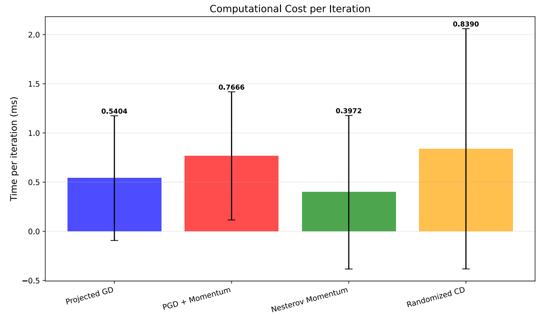


Fig. 7. $f(x_k)$ vs. k for all the methods ($\varepsilon = 10^{-8}$)

On Fig. 7, we see that the Projected Gradient method with the Barzilai-Borwein step surpasses the other methods by far. Then, the Nesterov's Accelerated Gradient Descent beats the last two. Again, we clearly see that the Randomized Coordinate Descent offers a way worse convergence rate compared to the other methods. It is also interesting to compare the elapsed time per iterations. Here is, like before, a graph showing the statistics about these:



	Mean [ms]	Std [ms]	Iterations
PGD + Adaptive step	0.2857	0.4518	7
PGD + Momentum	0.2826	0.4703	811
PGD + Nesterov	0.2531	0.4449	799
Randomized CD	0.2901	0.5484	1000

Fig. 8. Time per iteration statistics ($\varepsilon = 10^{-8}$, max_iter = 1000)

We see that PGD with Barzilai-Borwein step and the Nesterov's momentum methods offers the best cost-per-iteration values. One may choose between one of those two for this problem, as they share overall similar performances.

C. *Influence of λ on the solution*

We will now interpet the effect of the parameter λ on the smooth Markowitz model with our best algorithm (i.e Projected gradient with Barzilai-Borwein adaptive step).

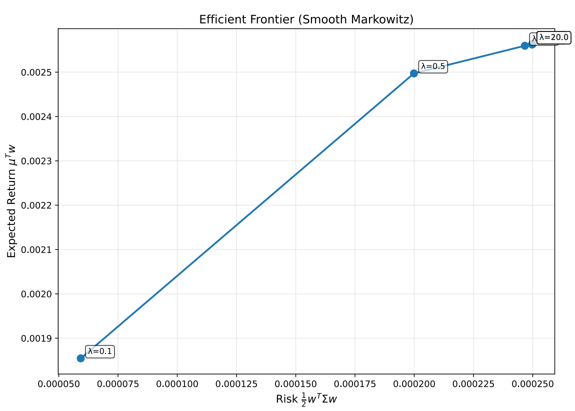


Fig. 9. Efficient frontier for the Smooth Markowitz Model

From what we see on these plots, the efficient frontier has a concave curve. We can interpret that as higher return also mean higher risk which directly translate reality. For initial values of $\lambda \in \{0.1, 0.5\}$, we have a low-risk but also a low return, increasing the λ directly yields a bigger return but also a bigger risk. Indeed, the bigger the lambda is the more we try to maximise our return. From the Efficient frontier we see that we have a saturation effect from $\lambda = 2$ to $\lambda = 20$. We also wanted to analyze the convergence speed with respect to λ:

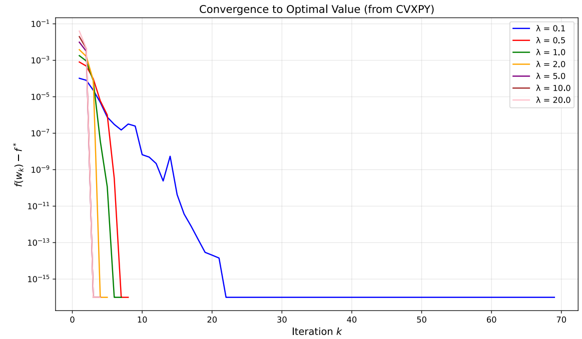


Fig. 10. $f(x_k) - f*$ vs k for different λ

In terms of convergence, we see that for bigger λ's, we converge faster. This is likely due to the fact that the second term becomes dominant and induces a steeper objective landscape.

D. *Non-Smooth model*

In this section, we will do similar numerical analysis, this time for the methods applied to the non-smooth model (2).

a) *Projected Subgradient descent:* We will start by analyzing the Subgradient method by first plotting the error $\|f(x_k) - f^*\|$ as a function of the iteration for different constant step sizes using the previously found rule $\alpha = \varepsilon / M^2$.

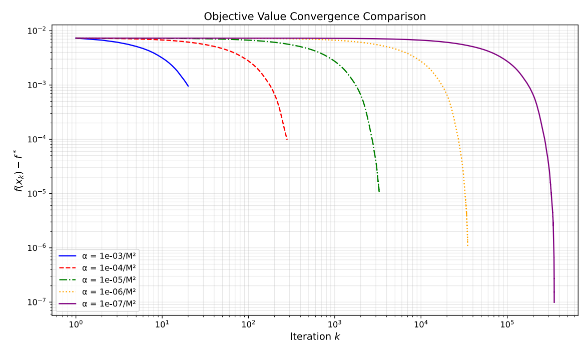


Fig. 11. $\|f(x_k) - f^*\|$ vs k for several Constant Step $\frac{\varepsilon}{M^2}$

As we can see on Fig. 11, each curve has the same shape and the larger ε is, the faster the convergence.

We also plotted the complexity curve here to verify that we observe numerically a complexity of $\mathcal{O}(\varepsilon^{-2})$:

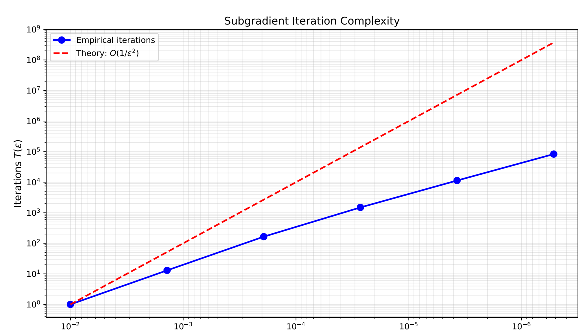


Fig. 12. Number of iterations vs. ε for the Projected Subgradient

By looking at the graph, we can confirm that the convergence rate is better than the theoretical one for this method.

Then we also implemented the diminishing step size for the projected subgradient and we plotted the error with the correct objectif value :

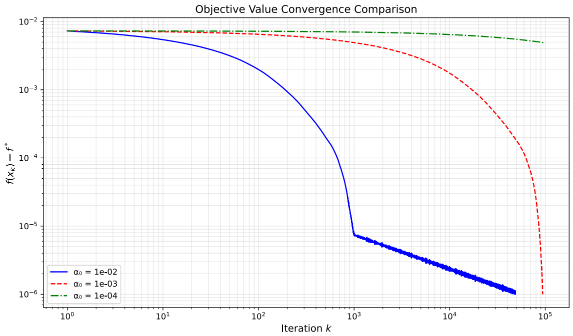


Fig. 13. $\|f(x_k) - f^*\|$ vs iteration for several Constant Step α (Tol :10^{-6})

We observe that only two of the three methods have converged, the one starting with an initial step $\alpha_0 = 0.01$ and the one with 0.001. Overall, we see that it does not outperform the constant step size version of this algorithm.

b) *Proximal gradient descent:*

For the proximal descent, we will first plot the error on the objective value as a function of the iterations. The reason why we did not choose a step size of $\frac{1}{L}$ is because the method converged in one iteration and it was thus not interesting to plot it.

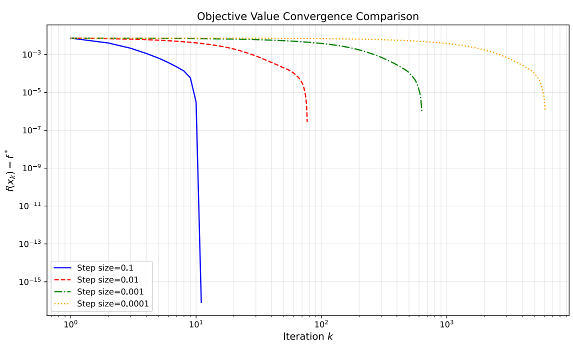


Fig. 14. $\|f(x_k) - f^*\|$ vs iteration for several step sizes

We directly see that for any step size we quickly converge towards the equilibrium which makes it for now the best method we have. We also implemented the Fast Proximal gradient which also converge in a single iteration for $t = 1/L$ so here is the comparison of the two objective function for a constant step size of 0.01:

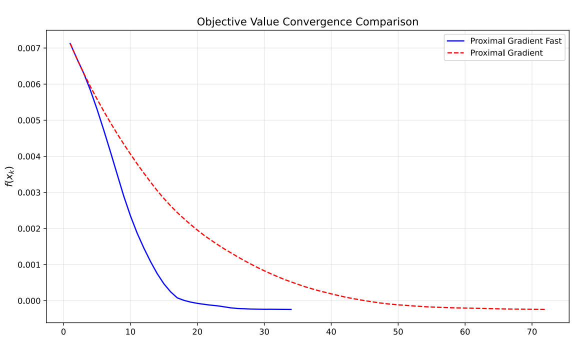


Fig. 15. $f(x_k)$ vs iteration for Fast and Classic Proximal Gradient

We then see that it converges in less iterations. However, the time per iteration is larger for the accelerated version:

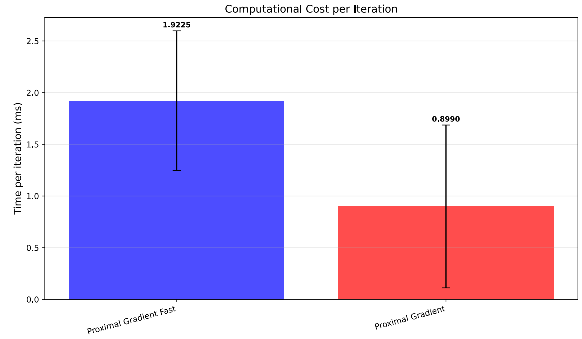


Fig. 16. Time per iteration for Fast and Classic Proximal Gradient

c) *Long-step Path-following Interior-Point method:*

Finally, we will perform a similar analysis for the Long-step Path-following Interior point method. Here we have first plotted the objective function:

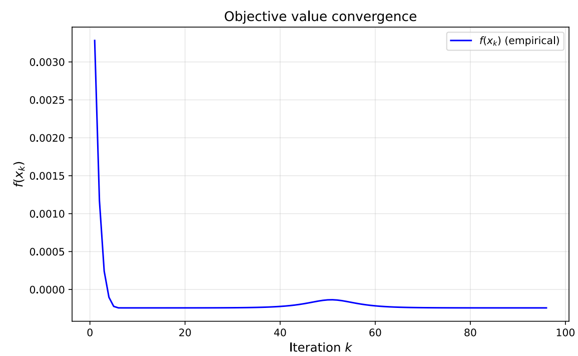


Fig. 17. $f(x_k)$ vs k

On Fig. 17, we can see that it quickly converges to the optimal value. However, even though the convergence rate is impressive, we have to take into account the cost-per-iteration which is quite high for this method:

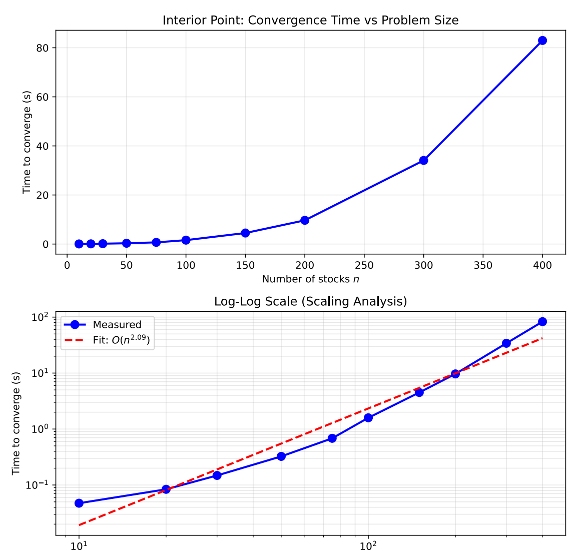


Fig. 18. Time per iteration (ms)

Here, we plotted the elapsed time to reach convergence for this method as the number of stocks n increases. We see that the time complexity is about $\mathcal{O}(n^2)$. In practice, this method takes too much time to be considered a suitable method for this problem.

E. *Influence of c on the solution :*

In this section we will analyze and interpet the influence of the parameter c, just like we did for λ for the previous model. As we mentionned previously, the parameter c controls the sensitivity with respect to the transaction costs. A higher c means that we consider larger transaction costs and vice-versa. Here we have plotted the number of assets for which we have a non-null weight for different value of the parameter c.

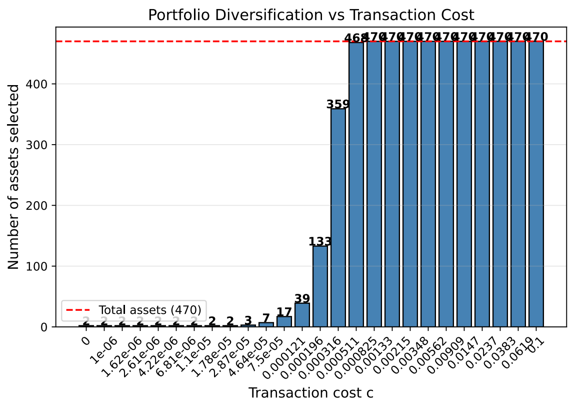


Fig. 19. Number of non-zero assets for several value of c

Here, we see that the higher the transaction cost, the higher the number of selected assets. This is due to the fact that we used a uniform reference portfolio, for which we do not want to get far away when c is large. When c is really small, we are allowed to change the portfolio considerably, which is why the model will try to invest in a single asset that looks promising.

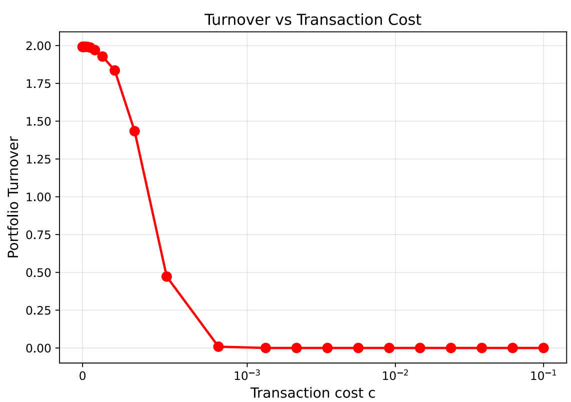


Fig. 20. Turnover vs c

On this graph, we also plotted the *turnover* of the portfolio. This essentially represents how much, in percentage, our portfolio changed with respect to our reference one. Again, we clearly see that this percentage drastically decreases as the transaction cost increases.

VI. Conclusion

To summarize, we have seen a various number of techniques in order to optimize the smooth and son-smooth version of the Markovitz problem. The two methods for both models that we could highlight as the best performing methods were respectively the Projected Gradient method with a Barzilai-Borwein adaptive step for the smooth model and the Proximal gradient descent for the non-smooth one. We also analyzed the effect of the parameters λ and c on the optimal solution and highlighted some interesting results, for instance the efficient frontier when λ varies and the optimum asset allocation as c, the transaction cost, increases.

References

[1] H. Markowitz, “Portfolio Selection,” *The Journal of Finance*, vol. 7, no. 1, pp. 77–91, 1952.

[2] “Karush–Kuhn–Tucker conditions.” [Online]. Available: https://en.wikipedia.org/wiki/Karush%E2%80%93Kuhn%E2%80%93Tucker_conditions

[3] “Subgradient Methods.” [Online]. Available: https://web.stanford.edu/class/ee364b/lectures/subgrad_method_notes.pdf

[4] “Newton's Method Constrained Optimization.” [Online]. Available: https://won-j.github.io/M1399_000200-2021fall/lectures/22-newton/newton_constr.html

[5] “Proximal Gradient Slides.” [Online]. Available: https://www.stat.cmu.edu/~ryantibs/convexopt/lectures/prox-grad.pdf

VII. Organization for the project

Groupe Number : 9
Students :

- Guerand Dewell: Contributed to numerical analysis of methods and implementation
- Lucas Ahou: Contributed to the redaction of the report and also to the implementation

A. Usage of LLMs

Here are the LLMs we used to help us in this project. Note that no AIs were used to write this report.
a) *Claude.ai:*

- Gave the project assignment for context and asked to write a directory, classe and method templates to have a modular structure.
- Using Claude Opus 4.5 as an agent in VSCode, helped for debugging purposes as well as for code structure

b) *DeepSeek & ChatGPT:*

- Gave the project assignment for context and asked multiple questions to summarize and interpret some methods for understanding them.
- Asked questions about convergence rates

B. GitHub Repository

To collaborate, we used a GitHub repository available at this link: https://github.com/DewellGuerand/LINMA2471---Project